洛谷网校

2026 LUOGU 非专业级别收容能力认证第一轮

（SCP-J1）入门级 C++语言试题

认证时间：2026 年 8 月 9 日 09:30~11:30

考生注意事项：

- 试题纸共有 12 页，满分 100 分。请在洛谷作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。
- 试题由洛谷网校学术组命制，欢迎报名洛谷网校第一轮课程。课程内容包含专题讲解、真题讲评与本试题讲评。https://class.luogu.com.cn/course/yugu26acs

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 64 位计算机中，下列 C++ 基本类型中（ ）的单个变量占用的内存最大。

A. int　　B. bool　　C. double　　D. char

2. Ubuntu 是常见的 Linux 操作系统。假设当前登录的用户为 luogu，当前所在的工作目录为 /home/luogu，使用（ ）命令，可以在 /home/sjtu/phd 下新建子目录 paper。

A. mv ./sjtu/phd/paper
B. mkdir ~/sjtu/phd/paper
C. mv /home/sjtu/phd/paper
D. mkdir ../sjtu/phd/paper

3. 阅读下面的代码，其中说法错误的是（ ）。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 stack <int> s1, s2;
04 int T, x;
05
06 int main() {
07     cin >> T;
08     while(T--) {
09         string op; cin >> op;
10         if(op == "in") {
11             cin >> x; s1.push(x);
12         }
```

```
13          else {
14              if(s2.empty()) {
15                  while(!s1.empty()) {
16                      s2.push(s1.top());
17                      s1.pop();
18                  }
19              }
20              cout << s2.top() << endl;
21              s2.pop();
22          }
23      }
24      return 0;
25  }
```

A. 代码中使用的 stack 是 STL 提供的栈的容器，栈的特点是先进后出。

B. 代码所实现的功能和队列的功能类似。

C. 输入为 6 in 1 out in 2 in 3 out out 时，输出为 1 3 2。

D. 若不对输入数据的操作类型做 in 和 out 数量的保证，代码存在运行时错误的风险。

4. $(1C)_{16}$、$(24)_{10}$、$(31)_{8}$ 和 $(11011)_{2}$ 中，最大的数的十进制表示为（ ）。

A. 28　　B. 30　　C. 25　　D. 27

阅读下面的材料，完成第 5-7 题。

在计算机底层，int 类型与 unsigned int 类型的加法运算在 ALU（算术逻辑单元）中的执行路径完全一致——ALU 仅对两个操作数进行二进制补码加法，得到相同的 32 位二进制和，并同时设置状态标志位（如进位标志 CF 和溢出标志 OF）。二者的差异仅在于如何解释这个二进制结果：若解释为有符号数（int），则按补码规则读取；若解释为无符号数（unsigned int），则按普通二进制数值读取，因此同一个二进制和可能对应不同的十进制值。当加法结果超出该类型的表示范围时，即发生溢出，ALU 并不会抛出异常或中断，而是直接截断高位，仅保留低 32 位作为结果，并更新标志位——对于无符号数溢出，ALU 将进位标志 CF 置 1（表示最高位产生进位），而对于有符号数溢出，ALU 将溢出标志 OF 置 1（表示符号位错误），但计算结果本身仍被保存为截断后的低 32 位，后续程序需根据类型和标志位自行判断是否发生了溢出，ALU 本身不对溢出做额外处理。

5. ALU 是算术逻辑单元，根据材料可以推断其属于（ ）的一部分。

A. CPU　　B. 外存　　C. 内存　　D. 操作系统

6. int 类型变量 x 的值为 -1，将其强制类型转换为 unsigned int 类型输出，输出的值为（ ）。

A. -1　　B. 2147483647　　C. 4294967295　　D. 4294967296

7. 下面代码的输出为（ ）。

```
01 #include <iostream>
02 int main() {
03     unsigned int a = 2147483648;
04     int b = 1234567890;
05     std::cout << int(a + b) << std::endl;
06 }
```

A. 3382051538　　B. -912915758

C. 1234567889　　D. 每一次运行输出可能不同

8. 下面的表格是图 G 的邻接表，关于图 G 说法错误的是（ ）。

1	2 3 4 5
2	1 3
3	1 2 6
4	1 5
5	1 4
6	3

A. 图 G 可能是无向图。　　B. 图 G 中度最大的结点为结点 1。

C. 图 G 中共有 2 个连通块。　　D. 可以使用 vector 实现图 G 的邻接表结构。

9. 下列代码的输出为（ ）。

```
01 #include <iostream>
02 using namespace std;
03 int main(){
04     int A = 1, B = 1;
05     int& a = A, b = B;
06     a = 2, b = 2;
07     std::cout << A << ' ' << B << std::endl;
08 }
```

A. 1 1　　B. 1 2　　C. 2 1　　D. 2 2

10. 给定一个长度为 n 的数列 $A=[a_1,a_2,\ldots,a_n]$，从中选择若干个元素，并保持它们在原数列中的相对顺序不变，组成一个新序列 $B=[b_1,b_2,\ldots,b_k]$。若该新序列满足从第二项起每一项都严格大于前一项（即 $b_1<b_2<\cdots<b_k$），则称 B 为原数列的一个上升子序列。在所有可能的上升子序列中，长度 k 达到最大值的那个，称为该数列的最长上升子序列（LIS），其长度记为最长上升子序列的长度。下列方法中，（　）不能正确求解一个序列的最长上升子序列。

A. 贪心。从第 1 个数开始选择，每次选取比上一个数更大的最小的数。

B. 搜索。通过深度优先搜索，枚举每一个数应该选哪一个。

C. 动态规划。用 $dp[i]$ 表示以 a_i 结束的最长上升子序列的长度。此时有 $dp[i]=\max(dp[j])+1$，其中 j 满足 $1\le j<i$ 且 $a_j<a_i$。

D. 动态规划。用 $dp[i]$ 表示长度为 i 的上升子序列中，结尾数值可以取得的最小值。枚举 a_j，每次二分查找到 $dp[i]$ 大于等于 a_j 的最小的 i，将 $dp[i]$ 设置为 a_j。

11. 哈夫曼编码是实现信息压缩的常见编码方式。假设有一组字符 {a,b,c,d,e,f}，对应的频率分别为 0.05,0.09,0.12,0.13,0.16,0.45。那么字符串 abcdef 共有（　）种可能的哈夫曼编码。

A. 8　　B. 16　　C. 32　　D. 64

12. 二叉树 T 的中序遍历为 CADBEFG，后序遍历为 CBDAFGE，则其前序遍历为（　）。

A. EACDBGF　　B. ECADBGF　　C. EACBDGF　　D. EACDFGB

13. 在 1 到 200 的正整数中，能被 3 或 5 整除，但不能被 7 整除的整数共有（　）个。

A. 79　　B. 80　　C. 81　　D. 93

14. 数字 1,2,3,4,5,6 排成一列，要求奇数之间的相对顺序保持不变，偶数之间的相对顺序也保持不变，则共有（　）种不同的排列。

A. 10　　B. 20　　C. 30　　D. 40

15. 大语言模型（LLMs）技术发展日新月异。关于 LLM 与 CCF 有关活动，说法不正确的是（　）。

A. CCF 在 NOI 冬令营等活动中开展了 LLM 协助编程的试点比赛。

B. CCF 主办了大模型能力认证 LMCC。

C. CCF SPP 系列活动中举办了一系列紧贴 LLM 的讲座。

D. CSP-J/S 第一轮中，选手可以使用 LLM 辅助完成试题。

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

（1）

```
01 #include <iostream>
02 using namespace std;
03
04 bool isPrime(int n) {
05     for(int i = 2; i < n; ++i) {
06         if(n % i == 0) {
07             return false;
08         }
09     }
10     return true;
11 }
12
13 int main() {
14     int n;
15     cin >> n;
16     for(int i = 2; i + 2 <= n; ++i) {
17         if(isPrime(i) && isPrime(i + 2)) {
18             cout << i << " " << i + 2 << endl;
19         }
20     }
21     return 0;
22 }
```

假设输入的 n 是 5 到 1000 之间的正整数，完成下面的判断题和单选题。

• 判断题

16. 当输入为 14 时，输出的所有整数之和为 44。（ ）

17. 将第 16 行的 `int i = 2` 修改为 `int i = 1`，程序输出不变。（ ）

18. 无论输入为多少，输出的所有整数均为奇数。（ ）

• 单选题

19. 当输入为 50 时，输出的所有整数之和为（ ）？

A. 220　　B. 224　　C. 227　　D. 230

20. 将第 5 行的 `i < n` 修改为以下哪个时，程序输出不变？（ ）？

A. `i <= n`　　B. `i * i < n`

C. `i * i <= n`　　D. `i * i * i < n`

（2）

```
01 #include <iostream>
02 #include <string>
03 #include <vector>
04 #include <algorithm>
05 using namespace std;
06 int main() {
07     string a, b;
08     cin >> a >> b;
09     int n = a.size(), m = b.size();
10     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
11     for(int i = 0; i <= n; ++i) {
12         for(int j = 0; j <= m; ++j) {
13             if(i == 0) {
14                 dp[i][j] = j;
15             }
16             else if(j == 0) {
17                 dp[i][j] = i;
18             }
19             else if(a[i - 1] == b[j - 1]) {
20                 dp[i][j] = dp[i - 1][j - 1] + 1;
21             }
22             else {
23                 dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1;
24             }
25         }
26     }
27     cout << dp[n][m] << endl;
28     return 0;
29 }
```

假设输入的 a, b 是长度不超过 1000 的仅包含小写字母的字符串，完成下面的判断题和单选题。

• 判断题

21. 若交换输入的 a 和 b，程序的输出一定不变。（　）

22. 程序的输出一定小于等于 $n + m$。（　）

23. 程序的输出一定大于 $\min\{n, m\}$。（　）

• **单选题**

24. 当输入为 `aba bab` 时，程序的输出为（　）。

A. 4　　B. 5　　C. 6　　D. 7

25. 当输入为 `abcddd acbded` 时，程序的输出为（　）。

A. 7　　B. 8　　C. 9　　D. 10

26. 假设输入的 a, b 都是长度为 2 且仅包含 `abcd` 四种字母的字符串。有多少种不同的有序字符串对 a, b 使得程序的输出为 3？（　）

A. 16　　B. 72　　C. 84　　D. 156

（3）

```
01 #include <iostream>
02 #include <string>
03 #include <algorithm>
04 using namespace std;
05 int n;
06 string s;
07 int dfs(string t, int lst) {
08     int ret = 0;
09     for (int i = 0; i <= n; i++)
10         if (t[i] != '1')
11             ret = 1e9;
12     if (ret == 0)return 0;
13     for (int i = 0; i <= n; i++)
14         if (i != lst && s.substr(n - i, i) == t.substr(0, i)) {
15             string cur = t;
16             cur[i] ^= 1;
17             ret = min(ret, dfs(cur, i) +1);
18         }
19     return ret;
20 }
21 int main() {
22     cin >> s;
23     n = s.size();
24     string t0(n + 1, '0');
25     cout << dfs(t0, -1);
26     return 0;
```

```
27 }
```

假设若无特殊说明，输入的字符串 s 是一个长度不超过 18 的非空 01 串，完成下面的判断题和单选题。

• 判断题

27. 将第 11 行的 1e9 换成 1234567，程序的输出结果不变。（ ）

28. 将第 14 行的 if 语句中，i!=lst && 去掉，程序的输出结果不变。（ ）

• 选择题

29. 下列说法中正确的是（ ）。

A. 设输入 10011101010 和 01100010101 得到的结果分别为 a,b，那么 $a<b$。

B. 存在一种输入，使得程序发生自然溢出。

C. 把第 16 行改成 cur[i] = 'a' - cur[i];，程序的输出结果会改变。

D. 前三个选项都不对。

30. 假设输入为 00000001，那么程序的输出为（ ）？

A. 341　　B. 342　　C. 343　　D. ABC 均错

31. 假设输入为 1101110111110111110，那么函数 dfs 将被调用（ ）次。

A. 2^{18}　　B. 2^{19}　　C. 2^{20}　　D. ABC 均错

32. （4 分）假设输入为 0010001000010000001，那么程序的输出为（ ）？

A. 21　　B. 289746　　C. 525739　　D. ABC 均错

三、完善程序（单选题，每小题 3 分，共计 30 分）

（1）（序列第 k 小）给出一个长度为 n 的数列 $a_1,a_2,\ldots,a_n$ 和一个正整数 k，你需要输出第 k 小的数字。输入保证 $1\le k\le n\le 10^5$，$1\le a_i\le 2\times 10^9$，且 a_i 均为整数。

下面的程序采用二分答案的方法，通过二分法找到最小的 x 使得至少 k 个数字不超过 x。试补全程序。

```
01 #include <iostream>
02
03 using namespace std;
04
05 int n, k, a[100'005];
06 bool check(int x) {
07     int c = 0;
08     for (int i = 1; i <= n; i++)
09         if ( ① )
10             c++;
11     return ②;
```

```
}
int main() {
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    int l = 1, r = ③;
    while (l < r) {
        int x = ④;
        if (check(x))
            r = x;
        else
            ⑤;
    }
    cout << l << endl;
    return 0;
}
```

33. ① 处应填（ ）

A. a[i]<=x　　B. a[i] < x

C. x <= a[i]　　D. x < a[i]

34. ② 处应填（ ）

A. c <= k　　B. c < k

C. k <= c　　D. k < c

35. ③ 处应填（ ）

A. n　　B. a[n]

C. 1 << 32　　D. 2e9

36. ④ 处应填（ ）

A. (l + r) / 2　　B. (l + r + 1) >> 1

C. l + (r - l) >> 1　　D. l + (r - l) / 2

37. ⑤ 处应填（ ）

A. l = x + 1　　B. l = x

C. break　　D. l += x

（2）（走迷宫）有一个 $n \times m$ 的网格迷宫。记第 i 行第 j 列的格子为 (i,j)。迷宫由空地（用 `.` 表示）和墙壁（用 `#` 表示）组成。起点是 $(1,1)$，终点是 (n,m)，保证起点和终点都是空地。小 R 希望使用两种操作从起点到达终点：

- **步行**：移动到上下左右相邻的空地。
- **传送**：移动到上下左右距离为 2 的空地。中间跨越的格子可以是空地也可以是墙壁。

小 R 最多使用 k 次传送。请计算她到达终点的最少操作次数。如果无法到达，输出 -1。

输入保证 $1 \le n,m \le 100$，$0 \le k \le 10$。输入的 `grid[i][j]` 只有 `.` 和 `#` 两种字符。

下面的程序使用了广度优先搜索的方式完成本题。试补全程序。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 105, MAXK = 15;
04 const int dx[4] = ①;
05 const int dy[4] = {0, 0, -1, 1};
06 int n, m, k, dis[MAXN][MAXN][MAXK];
07 char grid[MAXN][MAXN];
08 struct Node {
09     int x, y, k;
10 };
11 int bfs() {
12     memset(dis, 0x3f, sizeof(dis));
13     queue<Node> q;
14     q.push({1, 1, 0});
15     dis[1][1][0] = 0;
16     while( ② ) {
17         Node u = q.front();
18         q.pop();
19         if(u.x == n && u.y == m) {
20             return ③ ;
21         }
22         for(int i = 0; i < 4; i++) {
23             int nx = u.x + dx[i];
24             int ny = u.y + dy[i];
25             if(nx >= 1 && nx <= n && ny >= 1 && ny <= m &&
grid[nx][ny] != '#') {
```

```
                if(dis[nx][ny][u.k] > dis[u.x][u.y][u.k] + 1) {
                    dis[nx][ny][u.k] = dis[u.x][u.y][u.k] + 1;
                    q.push({nx, ny, u.k});
                }
            }
        }
        if( ④ ) {
            for(int i = 0; i < 4; i++) {
                int nx = u.x + dx[i] * 2;
                int ny = u.y + dy[i] * 2;
                int nk = u.k + 1;
                if(nx >= 1 && nx <= n && ny >= 1 && ny <= m &&
grid[nx][ny] != '#') {
                    if(dis[nx][ny][nk] > dis[u.x][u.y][u.k] + 1) {
                        dis[nx][ny][nk] = dis[u.x][u.y][u.k] + 1;
                        ⑤;
                    }
                }
            }
        }
    }
    return -1;
}

int main() {
    cin >> n >> m >> k;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            cin >> grid[i][j];
        }
    }
    cout << bfs() << endl;
    return 0;
}
```

38. ① 处应填（　）？

A. {0, 0, -1, 1}　B. {0, -1, 1, 0}　C. {-1, 1, 0, 0}　D. {-1, 1, -1, 1}

39. ② 处应填（　）？

A. !q.empty()　B. !q.size()　C. !q.clear()　D. q.front()

40. ③ 处应填（　）？

A. dis[u.x][u.y][u.k] + 1　B. dis[u.x][u.y][u.k]

C. dis[u.x][u.y][k]　D. dis[u.x][u.y][0]

41. ④ 处应填（　）？

A. u.k == k　B. u.k > 0　C. u.k <= k　D. u.k < k

42. ⑤ 处应填（　）？

A. q.push(nx, ny, nk)　B. q.push_back({nx, ny, nk})

C. q.insert({nx, ny, nk})　D. q.push({nx, ny, nk})

广告 祝贺洛谷计划学员在 NOI2026 获得 71 枚奖牌（7 金 41 银 23 铜）的好成绩

1．第一轮（初赛课程）https://class.luogu.com.cn/course/yugu26acs

该计划讲授了第一轮测试的题型和常考内容，并提供模拟赛和讲评用于查缺补漏。对于希望熟悉第一轮考点、提升第一轮能力的同学均可报名。**本套试题的讲评，以及课程回放在报名后均可提供。**

报名该课程的学员，报名 2026 年所有洛谷计划/训练营课程都可以减 300 元！

2．洛谷语言入门计划/基础算法计划

针对 NOIP/CSP，以及 GESP 的基础算法进阶课程，包括课堂讲授与实验、课后练习答疑解惑与考评环节，完善语言算法知识体系。



了解更多请访问：https://www.luogu.com.cn/discuss/1294802

3．基础提高衔接计划

计划包括集中授课、题单作业布置、定期模拟比赛讲评，巩固算法基础和举一反三能力，**目标 CSP-J 高分，或者是通过 GESP 6-8 级**，为提高级打基础。从暑假一直学到 11 月！

- A 组（冲刺 CSP-J）：https://class.luogu.com.cn/course/yugu26xjna
- B 组（备战 GESP 6-8）：https://class.luogu.com.cn/course/yugu26xjnb

2024 年参与学员中，J 组一等人数多于二等多于三等。洛谷的王牌课程之一。

4．洛谷秋令营（基础组·提高组）9 月公开

面向已经掌握基础/进阶算法学员，通过讲题和模拟增加经验，提升 CSP J/S 应试能力。**只要数百元。**

请关注公众号获得最新的课程资讯。

2026 LUOGU 非专业级别收容能力认证第一轮

（SCP-J1）入门级 C++语言试题 参考答案

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

题号	1	2	3	4	5
答案	C	D	C	A	A
题号	6	7	8	9	10
答案	C	B	C	C	A
题号	11	12	13	14	15
答案	C	A	B	B	D

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

题号	16	17	18	19	20
答案	T	F	T	B	C

题号	21	22	23	24	25	26
答案	T	T	F	A	B	D
题号	27	28	29	30	31	32
答案	T	F	D	A	B	D（4 分）

三、完善程序（单选题，每小题 3 分，共计 30 分）

题号	33	34	35	36	37
答案	A	C	D	D	A
题号	38	39	40	41	42
答案	C	A	B	D	D